

Air Port Runway Management system

Final Project Report

Course CSE123 — Data Structures and Algorithms

Term Spring 2026

Faculty Faculty of Engineering, Egypt University

Team Members

Islam Tamer Soliman	24-101055
Omar Mohamed Ismail	24-101150
Omar Samir	24-101136
Hussein Mohamed Amer	24-101066
Youssef Elmaghraby	24-101114

MAX-HEAP

FIFO QUEUE

Two data structures implemented from scratch

Table of Contents

Abstract

1. Problem Statement

2. System Architecture

3. Data Structures from Scratch

3.1 MaxHeap

3.2 Queue

4. Application Layer

4.1 Flight and the Priority Formula

4.2 Runway State Machine

4.3 Scheduler

4.4 Command-Line Interface

5. Theoretical Complexity Analysis

6. Empirical Performance Analysis

7. Test Scenarios and Validation

8. Sample Run

9. Limitations and Possible Extensions

10. Conclusion

Appendix A — File-by-File Inventory

Appendix B — Build and Run Instructions

Abstract

This project simulates the central control loop of a small airport. A stream of inbound and outbound flights register with the system; the controller picks the most urgent one once a minute; and a fleet of parallel runways serves them in first-cleared-first-served order until takeoff or landing is complete. The urgency of a flight depends on its emergency status, fuel level, and how long it has been waiting. Two data structures — both implemented entirely from scratch — power the entire simulation: a binary **MaxHeap** for global priority selection and a linked-list **FIFO Queue** for per-runway service lines and the global flight log. The system is exercised by a 63-test unit-test suite and an interactive command-line simulator. Empirical measurements confirm the theoretical asymptotic bounds within a small constant factor across five orders of magnitude ($n = 10^2$ to 10^6).

1. Problem Statement

A modern airport has a finite number of runways and an effectively infinite stream of aircraft asking to use them. The decision the control tower must make every minute is: **which aircraft goes next, and on which runway?** That decision must respect three competing pressures:

- An aircraft declaring an **emergency** must be cleared before any routine traffic.
- An aircraft running **low on fuel** is on a hard physical clock and must be prioritised over equally-routine traffic that is full of fuel.
- All else being equal, the aircraft that has been **waiting the longest** should go next — fairness and total-wait time matter.

Once an aircraft is cleared to a specific runway the rule changes: that runway services its queue strictly in **first-cleared-first-served** order, because in-air sequencing constraints make it dangerous to reorder aircraft already approaching. The airport also maintains an audit log of every completed operation.

The two queueing models — global priority before clearance, FIFO after clearance — map cleanly onto a **max-heap** and a **linked-list queue** respectively. Implementing both data structures from scratch is the core technical requirement.

2. System Architecture

The project is organised in three layers, each depending only on the ones below it:

Layer 3 — CLI Front-end	<code>src/main.cpp</code>	Parses commands, calls Scheduler, prints results.
Layer 2 — Application Logic	<code>src/Scheduler.{h,cpp}</code> <code>src/model/*</code>	Flight + priority formula. Runway state machine. Scheduler: pending heap, runway list, global log.
Layer 1 — Data Structures	<code>src/ds/MaxHeap.h</code> <code>src/ds/Queue.h</code>	Generic, hand-implemented, allocation-managed.

Data Flow per Simulated Minute

```

registerFlight(F)
    |
    v
+-----+
| MaxHeap | age every flight +1 min (uniform shift, heap order preserved)
| (pending) |
+-----+
    | pop one (most urgent)
    v
pickRunway()      <-- least-loaded runway selection
    |
    v
+-----+
| Runway[i] | tick(1) advances service; completes flight when remaining = 0
| +-----+ |
| | Queue | |
| +-----+ |
+-----+
    | drain completions
    v
+-----+
| Queue | global flight log (accessed by the log command)
| (completed) |
+-----+

```

Every arrow above uses one of the two hand-written data structures.

3. Data Structures from Scratch

Both structures are templated on the value type so a single implementation handles every use case in the project (plain `int` in unit tests, `Flight` objects in the application).

3.1 MaxHeap<T>

Source: `src/ds/MaxHeap.h` (153 lines)

A binary max-heap stored in a contiguous array. The largest element (by `operator<`) is always at index 0.

Index arithmetic (zero-based):

```

parent(i) = (i - 1) / 2
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2

```

Operations:

Operation	Method	Time Complexity
Insert	<code>push(value)</code>	$O(\log n)$
Extract maximum	<code>pop()</code>	$O(\log n)$

Peek maximum	top()	O(1)
Check empty	empty()	O(1)
Bulk load	MaxHeap(vector<T>)	O(n)
Restore order	rebuild()	O(n)

push (sift up): the new element is appended at the end and swapped up while it is larger than its parent. Longest chain = $\text{floor}(\log_2 n)$ swaps.

pop (sift down): the root is saved, the last element is moved to the root, and the result is sifted down by repeatedly swapping with its larger child. Same height bound — $O(\log n)$.

buildHeap (Floyd's bottom-up): instead of n successive push calls ($O(n \log n)$), the array is given as-is and siftDown is called on every internal node from the last up to the root. Most calls terminate quickly because most nodes are leaves; total work is bounded by $2n$ — strictly **$O(n)$** .

rebuild() / rawMutable(): after flights are mutated in place (e.g. upgraded to emergency), rebuild() restores the heap property in $O(n)$ via the same Floyd algorithm.

Source code excerpt — siftDown:

```
void siftDown(int i) {
    int n = data.size();
    while (true) {
        int l = left(i), r = right(i), largest = i;
        if (l < n && data[largest] < data[l]) largest = l;
        if (r < n && data[largest] < data[r]) largest = r;
        if (largest == i) break;
        swap(data[i], data[largest]);
        i = largest;
    }
}
```

3.2 Queue<T>

Source: src/ds/Queue.h (148 lines)

A FIFO queue built from a singly-linked list. Two pointers (head and tail) plus a cached count.

Layout: head -> [v1] -> [v2] -> ... -> [vn] <- tail

Operations:

Operation	Method	Time Complexity
Enqueue	enqueue(v)	O(1)
Dequeue	dequeue()	O(1)
Peek front	front()	O(1)
Peek back	back()	O(1)
Size	size()	O(1)
Clear	clear()	O(n)

Rule of Three: the class manages raw heap-allocated Nodes, so the destructor, copy constructor, and copy assignment operator are all explicitly defined. Without them, copying a Queue would shallow-copy head and tail pointers, and the second destructor would double-free the node chain. The CLI's `logListString()` and `pendingListString()` helpers rely on deep copies to produce non-destructive views of the live structures.

Source code excerpt — enqueue / dequeue:

```
void enqueue(T value) {
    Node* node = new Node(value);
    if (tail == NULL) { head = node; tail = node; }
    else { tail->next = node; tail = node; }
    count++;
}

T dequeue() {
    if (head == NULL) { cerr << "[Queue] empty"; return T(); }
    Node* old = head;
    T result = old->value;
    head = old->next;
    if (head == NULL) tail = NULL;
    delete old;
    count--;
    return result;
}
```

4. Application Layer

4.1 Flight and the Priority Formula

Source: `src/model/Flight.{h,cpp}`

A Flight carries: identifier, airline name, aircraft type, direction (ARR/DEP), fuel level (clamped 0..100%), accumulated waiting time, and an emergency flag.

Priority formula:

```
priority(f) = f.waitingTime
            + (f.fuelLevel < LOW_FUEL_THRESHOLD
               ? (LOW_FUEL_THRESHOLD - f.fuelLevel) * FUEL_FACTOR
               : 0)
            + (f.emergency ? EMERGENCY_BONUS : 0)

Constants:
    EMERGENCY_BONUS    = 10,000
    LOW_FUEL_THRESHOLD = 30 (%)
    FUEL_FACTOR        = 200
```

Constants are chosen so each rule dominates every realistic combination of the rules below it: an emergency adds 10,000 — even a flight at 0% fuel waiting 3,000 minutes scores only 9,000, which is less. A flight at 0% fuel adds $30 \times 200 = 6,000$ — well above any realistic wait time alone.

4.2 Runway State Machine

Source: `src/model/Runway.{h,cpp}`

Each Runway owns two `Queue<Flight>` objects: **waiting** (flights cleared to this runway) and **completed** (flights just finished). State includes: busy flag, remaining minutes, current flight, totalOps.

tick(int minutes) — for each minute: (1) if idle and waiting is non-empty, dequeue the front flight and start serving it; (2) if busy, decrement remaining; if it reaches 0, move the flight to completed and become idle. Step 1 precedes step 2 deliberately — a single tick can start the next flight if the runway just became idle. This is a classic **producer-consumer** pattern: the Scheduler produces into waiting; the Runway consumes and produces into completed; the Scheduler then consumes from completed.

4.3 Scheduler

Source: `src/Scheduler.{h,cpp}`

The Scheduler ties everything together. Its private state is exactly four things:

```
MaxHeap<Flight> pending;    // every flight not yet cleared
vector<Runway>  runways;    // physical runways
Queue<Flight>   log;         // global completed-flight log
int             currentTime;
```

Every tick(1) performs four steps:

1. **Age every pending flight +1 minute.** Because the same +1 is added to every priority, relative ordering is unchanged and heap property is preserved. Cost: $O(n)$.

2. **Dispatch one flight at most.** Pop the heap top and clear it to the least-loaded runway (load = waitingQueueSize + remaining if busy). Ties go to the lowest-indexed runway. One dispatch per minute models real ATC bandwidth.

3. **Tick every runway by 1 minute.** Each runway independently advances its state machine.

4. **Drain completions into the global log.** Flights now in any runway's completed queue are moved into the Scheduler's log.

declareEmergency(flightNumber): walks the heap's underlying array via `rawMutable()`, sets the emergency flag on the matching flight, and calls `rebuild()` to restore heap order. Total cost: $O(n)$ search + $O(n)$ rebuild = **$O(n)$** .

4.4 Command-Line Interface

Source: `src/main.cpp`

A read-eval-print loop on top of Scheduler. Two default runways (RW-09L and RW-27R, service time 3) are created at startup.

Command	Effect
help	List every available command
addrunway <name> <svcTime>	Add a named runway with given service time
add <fNo> <airline> <type> <ARR DEP> <fuel> [emergenc	]Register a flight
emergency <flightNo>	Promote a pending flight to emergency status
tick <minutes>	Advance the simulation by N minutes
run	Auto-tick until heap empty and all runways idle
status	Snapshot of every runway, pending count, log size
pending	List pending flights in priority order (non-destructive)
log	List completed flights in FIFO order (non-destructive)
quit / exit	Exit the simulator

5. Theoretical Complexity Analysis

Per-operation costs:

Operation	Cost	Reason
MaxHeap::push	$O(\log n)$	Sift up at most height-of-heap levels.
MaxHeap::pop	$O(\log n)$	Sift down at most height-of-heap levels.
MaxHeap::top	$O(1)$	Direct array index access.
MaxHeap::buildHeap	$O(n)$	Floyd's bound.
MaxHeap::rebuild	$O(n)$	Same Floyd algorithm.
Queue::enqueue	$O(1)$	Tail-pointer append.
Queue::dequeue	$O(1)$	Head-pointer remove.
Queue::front / back	$O(1)$	Pointer dereference.
Queue::size	$O(1)$	Cached counter.
Queue::clear / destructor	$O(n)$	Walk and free every node.
Queue copy ctor / assign	$O(n)$	Deep copy of linked list.

Per-tick cost of the simulator:

Step	Cost
Age every pending flight by +1	$O(n)$
One heap pop	$O(\log n)$
Tick every runway by 1 minute	$O(R)$
Drain completions on every runway	$O(R)$
Total per simulated minute	$O(n + R)$

For realistic airport sizes (n in the hundreds, $R < 10$) this is trivial — the simulator is real-time even at orders of magnitude beyond what any human-driven CLI could keep up with.

Floyd's Heapify Lower Bound (sketch):

A binary heap of size n has at most $\text{ceil}(n / 2^{k+1})$ nodes at height k , and a siftDown from height k does at most k swaps. The total number of swaps is:

$$\begin{aligned}
 & \sum_{k=0}^{\log n} \text{ceil}(n / 2^{k+1}) * k \\
 & \leq n * \sum_{k=0}^{\infty} k / 2^{k+1} \\
 & = n * 1 = n
 \end{aligned}$$

Hence buildHeap is **$O(n)$** , strictly better than n successive push calls ($O(n \log n)$).

6. Empirical Performance Analysis

Measurements taken with `g++ -std=c++17 -O2` using `std::chrono::steady_clock`. Random keys were pre-generated outside the timed region. Five sizes: 100 / 1,000 / 10,000 / 100,000 / 1,000,000.

Operation	n	Total Time	Per-op
MaxHeap push	100	20 μ s	0.200 μ s/op
MaxHeap pop	100	16 μ s	0.160 μ s/op
MaxHeap buildHeap	100	1 μ s	0.010 μ s/op
Queue enqueue	100	7 μ s	0.070 μ s/op
Queue dequeue	100	3 μ s	0.030 μ s/op
MaxHeap push	1,000	22 μ s	0.022 μ s/op
MaxHeap pop	1,000	52 μ s	0.052 μ s/op
MaxHeap buildHeap	1,000	9 μ s	0.009 μ s/op
Queue enqueue	1,000	82 μ s	0.082 μ s/op
Queue dequeue	1,000	43 μ s	0.043 μ s/op
MaxHeap push	10,000	175 μ s	0.018 μ s/op
MaxHeap pop	10,000	706 μ s	0.071 μ s/op
MaxHeap buildHeap	10,000	111 μ s	0.011 μ s/op
Queue enqueue	10,000	482 μ s	0.048 μ s/op
Queue dequeue	10,000	315 μ s	0.032 μ s/op
MaxHeap push	100,000	1,942 μ s	0.019 μ s/op
MaxHeap pop	100,000	13,463 μ s	0.135 μ s/op
MaxHeap buildHeap	100,000	984 μ s	0.010 μ s/op
Queue enqueue	100,000	5,579 μ s	0.056 μ s/op
Queue dequeue	100,000	3,987 μ s	0.040 μ s/op
MaxHeap push	1,000,000	13,808 μ s	0.014 μ s/op
MaxHeap pop	1,000,000	133,587 μ s	0.134 μ s/op
MaxHeap buildHeap	1,000,000	9,765 μ s	0.010 μ s/op
Queue enqueue	1,000,000	50,905 μ s	0.051 μ s/op
Queue dequeue	1,000,000	33,587 μ s	0.034 μ s/op

- **MaxHeap::push matches $O(\log n)$.** Per-op cost is 0.014–0.022 μ s across all sizes — essentially flat because $\log n$ grows so slowly the constant factor dominates.
- **MaxHeap::pop matches $O(\log n)$ more dramatically.** A pop almost always drives the moved-up element back to a leaf. Per-op grows from 0.052 μ s at $n=1,000$ to 0.135 μ s at $n=1,000,000$ — roughly doubling each time n multiplies by 100 ($\log n / \log 100$ growth).
- **MaxHeap::buildHeap is empirically linear.** Per-op is ~ 0.010 μ s at every size. Total at $n=1,000,000$ is 9.8 ms vs. 13.8 ms for successive pushes. Floyd's algorithm working as advertised.

- **Queue operations are constant-time per operation.** Per-op stays 0.03–0.08 μs across all sizes with no upward trend. Total time scales linearly — almost exactly the 10,000 $\times$ factor from 100 to 1,000,000 elements.

7. Test Scenarios and Validation

The project ships **63 unit tests** in a hand-written, exception-free framework (`tests/test_framework.{h,cpp}`). An assertion failure stores a message in a global `TestState` and returns out of the test body; the runner inspects the flag after each test.

File	Tests	Coverage Area
<code>test_maxheap.cpp</code>	8	empty/size, push order, pop monotonicity, 1,000-element stress, Floyd's load, heap invariant.
<code>test_queue.cpp</code>	10	FIFO order, mixed enqueue/dequeue, copy ctor / assignment / self-assign, 10,000 stress.
<code>test_flight.cpp</code>	11	constructor/clamp/wait, priority for every constant, emergency-vs-fuel domination.
<code>test_runway.cpp</code>	13	idle/busy, single-flight timing, separation between consecutive flights, 100-flight stress.
<code>test_scheduler.cpp</code>	21	priority dispatch, rate limit, load balancing, ageing, emergency promotion, log views.
TOTAL	63	

Selected high-value test cases:

- **MaxHeap:** heap property holds at every internal node — After every operation, walks the array and verifies `data[i] >= data[2i+1]` and `data[i] >= data[2i+2]` for every internal node. Catches any bug in `siftUp` or `siftDown`.
- **Queue:** copy constructor performs a deep copy — Copies a queue, modifies the original, asserts the copy is unaffected, and verifies no double-free on destructor — the exact bug the Rule of Three prevents.
- **Flight:** an emergency always beats any reasonable non-emergency — Constructs an emergency flight with terrible parameters and a non-emergency with the most extreme realistic parameters (0% fuel, 9999 min wait) — emergency still wins.
- **Runway:** completion is followed by a one-minute separation — Pins down the discrete-time semantics of tick.
- **Scheduler:** dispatch goes to least-loaded runway — Verifies load-balancing policy and that ties pick the lowest-indexed runway.

8. Sample Run

The file `docs/demo_scenario.txt` contains a 13-line script that exercises every feature of the system. Replay with:

```
Get-Content docs\demo_scenario.txt | .\build\runway_sim.exe
```

Demo script:

```
addrunway RW-EXTRA 2
```

```
add MS999 Egyptair B777 ARR 80 emergency
add AF120 AirFrance A320 ARR 5
add BA456 BritAir B737 DEP 90
add LH100 Lufthansa A350 ARR 75
pending
tick 5
status
log
emergency BA456
run
log
quit
```

What happens step-by-step:

1. A third runway RW-EXTRA is added to the two pre-loaded ones.
2. Four flights are registered: an emergency, a low-fuel flight (5%), and two ordinary flights.
3. `pending` lists all four in priority-pop order: MS999 (10000) → AF120 (5000) → BA456 (0) → LH100 (0). This is a direct visual of the heap doing its job.
4. `tick 5` advances five minutes. With three runways and one dispatch per minute, four flights get cleared and three complete by minute 5.
5. `status` reports: 0 pending, RW-09L busy with LH100 (1 min left), the other two idle, log holds 3 completions.
6. `log` lists three completions in actual completion order — emergency first, then low-fuel, then ordinary.
7. `emergency BA456` returns "no pending flight" because BA456 is already in a runway queue — a documented limitation.
8. `run` auto-ticks until everything drains. `log` then shows all four flights in completion order.

9. Limitations and Possible Extensions

Current Limitations (deliberately disclosed):

- **Mid-flight emergency declaration.** Once dispatched to a runway, `declareEmergency` cannot promote a flight. Only flights still in the pending heap can be upgraded.
- **Heap stability.** Two flights with equal priority pop in unspecified order. In practice ties are very rare (every pending flight ages +1 each minute, so collisions only occur within the same minute).
- **Single-runway compatibility.** Every flight is assumed to be able to use every runway. A real airport would model runway length, aircraft size, etc.

Possible Extensions:

- **Decrease-key / increase-key in $O(\log n)$.** Exposing `siftUp(int i)` and `siftDown(int i)`, plus a `flight-number → index` map, would turn `declareEmergency` into a true $O(\log n)$ operation.
- **Taxiway routing graph.** A directed graph over runways, taxiways, gates, and terminals would let the simulator route aircraft on the ground after landing.
- **BST-indexed schedule.** Pre-published schedules stored in a BST keyed by time would catch potential conflicts at registration rather than at dispatch.

10. Conclusion

The Airport Runway Management System is built around a single decision made every simulated minute — which flight goes next, and on which runway? — and that decision is exactly the operation that a priority queue exists to perform. Layering a per-runway FIFO queue on top captures the post-clearance behaviour with equal precision. The two data structures, hand-implemented in fewer than 200 lines of header each, together support a correct, real-time, end-to-end simulator with a 63-test suite and a CLI demo.

The empirical measurements match the theoretical bounds within a small constant factor across five orders of magnitude, validating both the implementation and the analysis. The project deliberately avoids exceptions and namespaces, in keeping with the second-year C++ subset taught in the course. Its empty-access contract (check first; otherwise diagnostic + default return) gives the same safety guarantees without exception machinery. The Rule of Three on `Queue<T>` is exercised in production by the CLI's non-destructive view helpers, demonstrating that the discipline pays off the moment any structure managing raw resources needs to be copied.

Appendix A — File-by-File Inventory

File	Lines	Description
src/ds/MaxHeap.h	153	Binary max-heap, array-backed, templated.
src/ds/Queue.h	148	Linked-list FIFO with Rule of Three.
src/model/Flight.h	92	Flight data + priority interface.
src/model/Flight.cpp	86	Priority formula implementation.
src/model/Runway.h	56	Runway state machine interface.
src/model/Runway.cpp	99	tick / clearFlight implementation.
src/Scheduler.h	110	Central coordinator interface.
src/Scheduler.cpp	~190	tick / dispatch / log management.
src/main.cpp	~190	CLI REPL.
tests/test_framework.h	~95	Macro / function-pointer test runner.
tests/test_framework.cpp	~50	Registry + runAllTests.
tests/test_main.cpp	5	Entry point.
tests/test_maxheap.cpp	~140	Heap unit tests + stress.
tests/test_queue.cpp	~150	Queue unit tests + Rule of Three.
tests/test_flight.cpp	~140	Priority formula coverage.
tests/test_runway.cpp	~190	Runway state machine coverage.
tests/test_scheduler.cpp	~280	End-to-end scheduler coverage.
docs/final_report.md	—	This report (source).
docs/demo_scenario.txt	—	Canned CLI script.
build.bat	—	One-command build and test on Windows.
README.md	—	Project overview.

Appendix B — Build and Run Instructions

Prerequisites: MinGW g++ 14.2 or newer (any C++17 compiler works).

From a PowerShell or cmd window in the project root:

```
.\build.bat
```

This builds two binaries inside `build/`:

- `run_tests.exe` — runs the full 63-test suite.
- `runway_sim.exe` — interactive CLI simulator. If the test step fails, the simulator is not built.

Replaying the demo:

```
Get-Content docs\demo_scenario.txt | .\build\runway_sim.exe
```

If the test step fails, review the output: each failed test prints its name and the assertion that triggered. Fix the data structure, re-run build.bat. The simulator binary is intentionally withheld until all 63 tests pass.